

МАШИННОЕ ОБУЧЕНИЕ

для начинающих

Учебник для тех, кто хочет научить компьютер думать

Практический курс · от первых данных до своего проекта
Python · Pandas · Scikit-learn · нейросети · LLM

Содержание

Как читать эту книгу

Эта книга — не сборник формул и не толстый справочник, который страшно открывать. Это **маршрут**: от «я вообще не понимаю, что такое машинное обучение» до момента, когда ты запускаешь собственное приложение и отправляешь ссылку на него друзьям.

Ты будешь не читать про машинное обучение, а заниматься им. Почти в каждой главе есть код, который можно запустить прямо в браузере, и задание, которое стоит сделать самому.

Правило простое: если можно попробовать — попробуй.

Что нужно знать заранее

Мы предполагаем, что ты уже писал простые программы на Python: знаешь, что такое переменная, список, цикл и функция. Если ты когда-нибудь делал запрос к базе данных (SQL) — вообще отлично, это пригодится. Всё остальное объясняется с нуля.

Как устроена книга

Книга разбита на шесть частей — мы называем их фазами. Каждая следующая фаза опирается на предыдущую, поэтому читай по порядку.

Шесть фаз пути

Фаза 1 — Данные и ML-мышление. Учимся смотреть на данные и находить в них закономерности.

Фаза 2 — Классический ML. Строим первые модели, которые предсказывают.

Фаза 3 — Нейросети. Разбираемся, как машина «видит» и учится на примерах.

Фаза 4 — Большие языковые модели (LLM). Понимаем, как работает то, что стоит за ChatGPT.

Фаза 5 — AutoML и веб-приложения. Превращаем модель в продукт с кнопкой.

Фаза 6 — Финальный проект. Делаем и защищаем что-то своё.

Значки, которые встретятся в тексте

Аналогия

Так помечены сравнения из обычной жизни — они помогают понять сложную идею быстро.

Внимание, частая ошибка

Так отмечены места, где почти все спотыкаются. Прочитай внимательно — сэкономишь часы.

Полезно знать

Дополнение или уточнение, которое углубляет понимание, но не обязательно для первого прохода.

Где запускать код

Весь код в книге рассчитан на **Google Colab** — бесплатную среду, которая работает прямо в браузере. Ничего устанавливать не нужно: заходишь на colab.research.google.com, создаёшь новый блокнот и вставляешь код. Всё считается на серверах Google.

Готов? Тогда начнём с главного вопроса: чем машинное обучение вообще отличается от обычного программирования.

ФАЗА 1

Данные и ML-мышление

Учимся видеть закономерности в данных — фундамент всего, что будет дальше.

Глава 1. «Магия» машинного обучения

Две разные идеи: правила и обучение

Представь, что тебя попросили написать программу, которая отличает кошку от собаки на фотографии. Как бы ты это сделал обычным способом — через правила?

Ты бы начал: «Если уши треугольные и торчат вверх — это кошка. Если морда вытянутая — собака...» И очень быстро увяз бы. А вислоухая кошка? А чихуахуа с торчащими ушами? А фото сбоку? Правил становятся тысячи, и они всё равно не работают.

Машинное обучение переворачивает задачу. Вместо того чтобы писать правила, ты показываешь машине **тысячи примеров** — «вот кошка», «вот собака», «вот снова кошка» — и она сама находит закономерности, которые отличают одно от другого. Правила она выводит без тебя.

Аналогия: рецепт и дегустация

Обычное программирование — это рецепт: ты записываешь точные шаги, и повар их выполняет. Машинное обучение — это дегустатор, который попробовал сотни блюд и теперь на вкус угадывает, солёное оно или сладкое, хотя рецепта ему никто не давал.

Определение своими словами

Главная мысль главы

Машинное обучение — это способ научить компьютер решать задачу не через записанные правила, а через примеры, из которых он сам выводит закономерности (паттерны).

Что такое «паттерн в данных»

Паттерн — это устойчивая закономерность. Люди находят паттерны постоянно, даже не замечая. Ты знаешь, что после серых туч часто идёт дождь. Никто не давал тебе формулу «туча → дождь» — ты вывел её из наблюдений. Именно этим занимается модель машинного обучения, только с числами.

Например, в данных о жилье может прятаться паттерн: «чем больше площадь, тем выше цена». В данных о фильмах: «фильмы с рейтингом выше 8 чаще собирают большую кассу». Модель ищет такие связи и потом использует их, чтобы делать **предсказания** на новых данных.

Типы задач машинного обучения

Почти всё, что мы будем делать, сводится к трём большим типам задач. Разберём каждый на понятном примере.

1. Классификация — «к какой категории?»

Модель выбирает одну из нескольких категорий. Спам или не спам. Кошка, собака или птица. Положительный отзыв или отрицательный. Ответ — это метка (label).

2. Регрессия — «сколько?»

Модель предсказывает число. Какой будет цена квартиры. Сколько баллов наберёт ученик. Какая температура будет завтра. Ответ — это число на шкале, а не категория.

3. Кластеризация — «кто на кого похож?»

Модель сама, без подсказок, разбивает данные на группы похожих объектов. Например, находит среди покупателей группы со схожим поведением. Здесь никто заранее не говорит модели правильный ответ — она группирует сама.

Тип задачи	Вопрос	Пример ответа
Классификация	К какой категории?	спам / не спам
Регрессия	Сколько? Какое число?	цена = 4 200 000
Кластеризация	Кто на кого похож?	группа А, В, С

Внимание, частая ошибка

Классификацию и регрессию путают чаще всего. Запомни разницу по вопросу: если ответ — это категория (одна из нескольких), это классификация. Если ответ — число на непрерывной шкале, это регрессия. «Сдаст экзамен / не сдаст» — классификация. «Сколько баллов наберёт» — регрессия.

Обучение с учителем и без учителя

Классификация и регрессия — это обучение с учителем (supervised): в данных есть правильные ответы, на которых модель тренируется. Кластеризация — обучение без учителя (unsupervised): правильных ответов нет, модель ищет структуру сама. В этом курсе мы почти всё время работаем с обучением с учителем — оно нагляднее и понятнее в начале пути.

Попробуй сам: игра «Угадай правило»

Эту игру можно сыграть без единой строчки кода — с другом или всем классом. Она показывает суть машинного обучения буквально за пять минут.

1. Один игрок загадывает секретное правило, например: «число проходит, если оно чётное».
2. Второй игрок называет числа: 4 → проходит, 7 → не проходит, 10 → проходит...
3. С каждым ответом второй игрок уточняет свою гипотезу о правиле — ровно как модель уточняет закономерность на новых примерах.
4. Когда второй уверен — он называет правило. Угадал? Значит «обучение» удалось.

Ты только что сработал как модель машинного обучения: получал примеры с ответами и постепенно выводил скрытое правило. Именно это делает алгоритм — только с тысячами примеров и очень быстро.

Проверь себя

1. Чем машинное обучение отличается от программирования правилами? Объясни в одном предложении.
2. Приведи по одному своему примеру задачи классификации и задачи регрессии.
3. Задача «разбить учеников школы на группы по интересам, не зная заранее какие» — это какой тип обучения?

Глава 2. Данные вокруг нас: знакомство с Pandas

Машинное обучение начинается не с моделей, а с данных. Прежде чем что-то предсказывать, нужно научиться данные открывать, читать и понимать. Главный инструмент для этого в Python называется Pandas.

Аналогия: Pandas — это Excel внутри Python 💡

Если ты когда-нибудь работал с таблицей в Excel или Google Таблицах — ты уже почти понимаешь Pandas. Это тот же самый лист с колонками и строками, только вместо мышки ты управляешь им кодом. Кода бояться не нужно: команд немного, и они читаются почти как английские фразы.

Три главных слова: датасет, признак, целевая переменная

Датасет — это таблица с данными. Каждая **строка** — один объект (например, один пассажир корабля). Каждый **столбец** — одна характеристика объекта, её называют **признаком** (feature): возраст, пол, цена билета.

Среди столбцов часто есть один особенный — тот, который мы хотим предсказывать. Его называют **целевой переменной** (target). Например, «выжил пассажир или нет». Все остальные признаки — это подсказки, по которым модель будет угадывать целевую переменную.

	Возраст (признак)	Пол (признак)	Выжил (цель)
Пассажир 1	22	муж.	нет
Пассажир 2	38	жен.	да
Пассажир 3	26	жен.	да

Первый код: открываем датасет

Загрузим известный учебный датасет о пассажирах «Титаника» — мы будем возвращаться к нему на протяжении нескольких глав. Вставь этот код в новую ячейку Google Colab и запусти.

```
# Импортируем библиотеку Pandas под коротким именем pd
import pandas as pd

# Загружаем датасет "Титаник" прямо из интернета по ссылке
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url) # df — сокращение от DataFrame ("таблица")

# Показываем первые 5 строк таблицы
df.head()
```

Ожидаемый результат: таблица из 5 строк с колонками Name, Sex, Age, Survived и другими.

Три команды, чтобы понять любой датасет

Как только данные загружены, три команды дают полную картину. Запускай каждую в отдельной ячейке.

df.head() — как выглядят данные

Показывает первые несколько строк. Это первое, что делают всегда: буквально посмотреть глазами, что за таблица перед тобой, какие есть колонки и что в них лежит.

df.info() — что за колонки и нет ли пропусков

```
df.info()
```

Показывает список колонок, их тип (число или текст) и сколько значений заполнено. Если в колонке Age заполнено 714 значений из 891 — значит, 177 пропущено. Это важный сигнал, к которому мы вернёмся в главе 4.

df.describe() — статистика по числам

```
df.describe()
```

Для каждой числовой колонки показывает среднее, минимум, максимум и другие величины. Один взгляд — и ты знаешь, что средний возраст пассажира около 30 лет, а самому младшему меньше года.

Фильтрация и группировка: привет из SQL

Если ты знаком с SQL, следующие операции покажутся родными. Pandas умеет то же самое, что запросы к базе данных, только внутри Python.

Фильтрация — как WHERE

```
# Оставить только пассажиров женского пола
women = df[df["Sex"] == "female"]
women.head()
```

Это прямой аналог SQL-запроса `SELECT * FROM df WHERE Sex = 'female'`. Внутри квадратных скобок мы пишем условие, и остаются только строки, для которых оно истинно.

Группировка — как GROUP BY

```
# Средняя доля выживших отдельно для мужчин и женщин
df.groupby("Sex")["Survived"].mean()
```

Эта одна строка отвечает на настоящий вопрос: кто выживал чаще? Результат обычно поражает — среди женщин доля выживших намного выше. Это и есть паттерн в данных, найденный за секунду.

Внимание, частая ошибка ⚠

Легко перепутать одиночное = (присваивание) и двойное == (сравнение). В условии фильтра всегда двойное: `df["Sex"] == "female"`. Одиночное = здесь вызовет ошибку.

Домашнее задание

Зайди на сайт Kaggle (kaggle.com/datasets) и найди датасет на тему, которая тебе интересна: игры, музыка, спорт, кино. Загрузи его в Colab через `pd.read_csv` и ответь на три вопроса:

- Сколько в нём строк и столбцов? (подсказка: `df.shape`)
- Какие есть признаки и есть ли среди них пропуски? (`df.info()`)
- Найди один интересный факт с помощью `groupby` — например, среднее значение чего-то по категориям.

Глава 3. Визуализация и разведка данных (EDA)

Таблица из тысяч чисел ничего не говорит человеку напрямую. А один хороший график — говорит сразу всё. Процесс, когда мы разглядываем данные через графики и ищем в них закономерности, называется **разведочным анализом данных** (EDA, Exploratory Data Analysis). Это как осмотреться на новом месте, прежде чем действовать.

Три графика, которые нужны почти всегда

Гистограмма — как распределены значения

Гистограмма показывает, каких значений в данных много, а каких мало. Например, каких возрастов среди пассажиров больше всего.

```
import matplotlib.pyplot as plt # библиотека для графиков

# Гистограмма возрастов пассажиров
df["Age"].hist(bins=20)         # bins — на сколько столбиков делить
plt.xlabel("Возраст")
plt.ylabel("Сколько пассажиров")
plt.title("Распределение возрастов")
plt.show()
```

Ожидаемый результат: столбчатая диаграмма — видно, что больше всего пассажиров в возрасте 20–35 лет.

Диаграмма рассеяния (scatter plot) — связь двух признаков

Каждая точка — один объект. По одной оси один признак, по другой — второй. Так видно, связаны ли они. Если точки складываются в линию — связь есть.

```
# Есть ли связь между возрастом и ценой билета?
df.plot.scatter(x="Age", y="Fare")
plt.show()
```

Тепловая карта корреляций (heatmap) — что с чем связано

Показывает сразу для всех пар признаков, насколько сильно они связаны. Понадобится Seaborn — библиотека красивых графиков поверх Matplotlib.

```
import seaborn as sns

# Считаем корреляции только по числовым колонкам
corr = df.corr(numeric_only=True)

# Рисуем тепловую карту: чем ярче клетка, тем сильнее связь
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.show()
```

Что такое корреляция

Аналогия: качели

Корреляция — это число от -1 до +1, показывающее, как связаны два признака. +1: когда один растёт, другой тоже растёт (как рост и вес). -1: когда один растёт, другой падает (как цена билета и класс каюты, где 1-й класс дороже 3-го). Около 0: связи нет, ведут себя независимо.

Внимание, частая ошибка

Корреляция — это не причина! Если мороженое и солнечные ожоги растут вместе, это не значит, что мороженое вызывает ожоги. Просто оба зависят от жаркой погоды. Всегда спрашивай себя: нет ли третьей причины?

Главное правило EDA: график без вывода бесполезен

Построить график — половина дела. Вторая половина — сформулировать словами, что он показывает. Возьми за привычку: под каждым графиком писать вывод одним предложением. «На гистограмме видно, что большинство пассажиров — молодые люди 20–35 лет».

Мини-конкурс: найди самый необычный инсайт

Возьми свой датасет из прошлого задания и построй на нём три разных графика. Для каждого напиши вывод словами. Затем выбери самый неожиданный факт — то, чего ты не ожидал увидеть. Это и есть инсайт: то, что данные рассказали тебе, а ты не знал заранее.

Проверь себя

1. Какой график выберешь, чтобы понять, как распределён возраст в классе?
2. Корреляция между двумя признаками равна $-0,8$. Что это значит?
3. Почему нельзя говорить «признак А вызывает признак В» только на основании высокой корреляции?

Глава 4. Мини-проект «Расскажи о данных»

Пришло время собрать всё из первой фазы вместе. Это твой первый мини-проект — небольшое, но полное исследование, которое можно показать друзьям и родителям. Здесь мы также научимся справляться с двумя частыми проблемами реальных данных: пропусками и выбросами.

Проблема 1: пропущенные значения

Реальные данные почти всегда неполные. В «Титанике» у многих пассажиров не указан возраст. Модель не умеет работать с пустыми клетками, поэтому пропуски нужно как-то обработать.

```
# Сколько пропусков в каждой колонке?
df.isnull().sum()

# Вариант 1: удалить строки с пропусками (осторожно — теряем данные)
df_clean = df.dropna()

# Вариант 2: заполнить пропуски средним значением (чаще лучше)
df["Age"] = df["Age"].fillna(df["Age"].median())
```

Полезно знать

Заполнять пропуски медианой (серединным значением), а не средним, безопаснее: медиана не «съезжает» из-за нескольких очень больших или очень маленьких значений.

Проблема 2: выбросы

Выброс (outlier) — это значение, которое сильно выбивается из общего ряда. Например, если у одного пассажира указана цена билета в 10 раз выше всех остальных. Выбросы могут быть как ошибкой ввода, так и настоящим редким случаем. Найти их помогает график типа «ящик с усами» (boxplot).

```
import matplotlib.pyplot as plt

# Ящик с усами по цене билета: точки далеко от "ящика" — выбросы
df.boxplot(column="Fare")
plt.show()
```

Структура твоего мини-проекта

Оформи исследование как небольшую презентацию из трёх частей. На выступление хватит трёх минут.

1. О данных. Что за датасет, сколько строк и столбцов, какие признаки. Один слайд.
2. Что нашёл. 2–3 графика с выводами словами и главный инсайт. Один-два слайда.
3. Вывод. Один содержательный факт, который данные рассказали тебе. Один слайд.

Как оценивается мини-проект

Мини-проект оценивается по простой рубрике — она помогает понять, что важно, а не для отметки.

Критерий	Баллы	Что оценивается
Постановка	0–2	Понятно, о каких данных речь
Работа с данными	0–3	Обработаны пропуски и выбросы
Визуализация	0–2	Графики уместны и подписаны
Объяснение	0–3	Есть вывод словами и инсайт

Главная мысль фазы 1

Ты научился главному навыку любого специалиста по данным: взять сырую таблицу, привести её в порядок, разглядеть в ней закономерности и рассказать о них человеческим языком. Теперь мы готовы сделать следующий шаг — научить машину предсказывать.